

汇编复习

0.Intro

- 物理地址(20位)=段基址+偏移量=段基址×16+偏移量
- 逻辑地址=段基址: 偏移量

址与立
机器指
期,需
度都要
理地址
址概念
不同的
这三种
存器提
共有 5

址地址,
出现。
以以符号数

逻辑

地址

存储器寻址:

位移量 (8位/16位常量)

偏移量

基址 (BX/BP的内容)

变址 (SI/DI的内容)

} 变量

段基址 (没有可做文章的地方)

- 这几个概念在Onenote笔记中有详细比喻描述, 可以去看
- 8086是16位数据总线, 8088是8位

1.寄存器

- 数据(通用)寄存器: AX累加器, BX基址, CX计数, DX数据, 都是16位
 - AX=AL+AH, 每部分8位
- 段寄存器: CS代码段, SS堆栈段, DS数据段, ES附加段
 - 一个段对应一段连续的内存空间
- 地址指针寄存器: BX基址, SP堆栈, BP基址指针, SI源变址, DI目的变址
 - 提供偏移量的基址 (BX/BP) 和变址 (SI/DI)
 - 默认搭配: SP/BP-SS, 剩余-DS
- 标志寄存器/状态位:
 - 1. CF进位: 无符号数加减运算最高位进位置1, 位运算会把CF强行置0
 - 2. PF奇偶: 低8位有偶数个1置1
 - 3. AF辅助进位: 第3位(低4位)进位置1, 用于BCD码进位
 - 4. ZF零: 运算结果为0置1
 - 5. SF符号位: 补码运算结果符号位/复制MSB
 - 6. OF溢出: 有符号数(补码) 溢出置1, 既对应十进制计算结果在-128~127之外的情况。
 - 移位指令中, 如果最高位变化, 那OF就等于1。但加减法要看是不是两个同符号数相加变号的情况

7. TF单步跟踪：单步跟踪置1

-
- 8. IF中断：置0屏蔽掉可屏蔽中断，置1相应所有中断
-
- 9. DF串操作：置0地址递增，置1地址递减，应该不太重要
- 汇总：

DEBUG 双字符状态表示的含义			
标志位	含义	FLAG = 1	FLAG = 0
OF	溢出(是/否)	OV Overflow	NV Not overflow
DF	方向(减量/增量)	DN Down	UP UP
IF	中断(允许/关闭)	EI Enable Interrupt	DI Disable Interrupt
SF	符号(负/正)	NG NeGative	PL PLus
ZF	零(是/否)	ZR ZeRo	NZ Not Zero
AF	辅助进位(是/否)	AC Auxiliary Carry	NA Not Auxiliary
PF	奇偶(是/否)	PE Parity Even	PO Parity Odd
CF	进位(是/否)	CY CarrY	NC Not Carry

2.基本指令

2.1.寻址方式

2.1.1.寄存器寻址

- 操作数在寄存器中，速度快
- 例：MOV AX,BX

2.1.2.立即数寻址

- 操作数以立即数形式体现
- 速度仅次于寄存器寻址
- 仅能用于源操作数寻址，不能用于目的操作数
- 例：MOV AL,25

2.1.3.存储器寻址

- 操作数位于内存单元中
- 具体方式：
 -
 - 1. 直接寻址：直接用指令字段作为偏移量，默认用DS作为段基址
 - 与立即数的区别在于中括号
 - 例 MOV AX,[1000]，源操作数 [1000H] 为直接寻址，将DS:1000H的数据读入AX寄存器
 - 例 MOV VAR1,BL，目的操作数VAR1为直接寻址，VAR1的地址充当偏移量，计算目的物理地址
 -
 - 2. 间接寻址：偏移量由5个地址指针寄存器提供⁵⁵⁶⁶⁵
 - 例：ADD CL,[BX]，将DS中BX寄存器的值作为内存地址访问到的 8 位数据，与 CL寄存器中的当前值相加，并将结果存储回CL寄存器中
 -
 - 3. 基址/变址寻址：偏移量来自地址指针寄存器+基址分量/变址分量
 - 例：MOV AL,[BP]100H 源操作数为基址寻址，偏移量为BP的值（作为地址）+100H
 - 例：AND AL,[BX][SI]0020H，偏移量为BX的值+SI的值+0020H
 - 例：MOV AX [BX+立即数] 也是基址寻址

BX -> 寄存器中的数据值。[BX] -> BX 中的值当作偏移量，计算出地址后去访问内存得到的数据值，即代表给出存放地址的寄存器

2.1.4其他寻址

串操作/端口/隐含

I/O端口寻址就是IN/OUT这些，直接寻址用8位地址，立即数给出；间接寻址用16位地址，用AX/DX寄存器

2.2.基本指令系统

双操作数指令最多有一个内存操作数，两个操作数不能同时为存储器寻址，带中括号的就是内存操作数，比如 [BP]，或者 VAR1 这样的变量

单操作数指令不能有立即数

注意位数不匹配/不确定的错误，比如 POP AL,16位指令对应了8位寄存器；或者 ADD [0100H],64H，系统不知道应该ADD 8位的64H还是16位的0064H

- **MOV**: move src->dest
 - 不能把立即数直接给段寄存器，也不能段寄存器间/立即数间传送，比如 MOV DS,11H 或者 MOV DS,CS 或者 MOV VA1,VA2
- **XCHG**:exchange src&dest
 - 不能有段寄存器
- **LAHF/SAHF**:取/存标志位
 - 标志寄存器低8位<->AH
- **PUSH/POP**: 将SRC（必须16位）出入栈。入栈指针-2，出栈+2，因为栈是从高地址往下生长的；注意进制转换问题，这里+-2都是十六进制下的运算。
- **LEA**:src的地址->dest
 - SRC必须是内存操作数，dest必须是16位
- **LDS/LES**:
 - SRC的数据解释为地址指向的内存单元的数据->dest
 - 刚才地址+2(16bit)指向的内存单元的数据->DS/ES
 - SRC必须是内存单元，dest必须是16位通用寄存器
 - 例； LDS BX,[0100H]，假设 0100H 中的数据为08H, 0102H 的数据为15H，则(BX)=08H,(DS)=15H
- **ADD/SUB**:算术加/减
 - dest不能为立即数
 - add既可以是无符号数编码计算，也可以是补码计算，具体是哪一种由设计者自行决定
 - 注意减法都是DEST-SRC，也就是 SUB A,B 等同于A-B，不要搞反了
- **ADC/SBB**:带进位加法/带借位减法：CF是另一个加数/被减数
 - 用于运算>16位的情况
- **INC/DEC**: i++
 - 不影响CF
- **NEG**:求相反数
 - 实际上是0-操作数，不是直接取NOT，因此解释为补码
 - 溢出判别自己去看书P87，我也不知道重不重要
- **CMP**: 比较
 - 本质上就是不保留结果的SUB，通过标志位判断大小
 - 无符号数具体结果：
 - CF=0: dest>src
 - CF=1: dest<src(溢出)
 - 补码具体结果：
 - OF=SF: dest>src
 - OF!=SF: dest<src,
- **AND/OR/XOR/NOT**:按位与/或/异或/非
- **TEST**: 测试
 - 本质上就是不保留结果的AND，有点类似CMP和SUB的关系[^a1e6b7](#)
 - 可用于测试某些位是否为1
 - 配合条件转移指令实现分支/循环结构
- **SAL/SAR**: 算术左移/右移
 - SAL/SAR DEST,COUNT，即将DEST左移/右移count位
 - DEST不能为立即数
 - count为1可以直接用1表示，否则都只能用CL寄存器中转
 - 最后一个被移出的位进CF，左右移都是如此
 - 在算术右移中，需要填充的高位统一由原数最高位(符号位)填充，保证正负性不变，因为左右移都被解释为补码
- **SHL/SHR**: 逻辑左移/右移
 - SHL和SAL的操作基本一样
 - SHR和SAR的区别在于高位填充，SHR统一用0
 - 因为逻辑左右移被解释为无符号数
- **ROL/ROR**:循环左移/右移
 - 移动的最后一位复制进CF

- OF=新CF异或新MSB
- **RCL/RCR**:带进位循环左/右移
 - 将CF看作最高/低位参与到循环移动中
 - OF=新CF异或新MSB
- **DEBUG**:

和汇编学习有关的几个 DEBUG 命令

命令字符	功 能
?	显示 DEBUG 命令列表
q	退出 DEBUG
r	显示或设置寄存器值
d	显示默认或指定地址范围内存单元的值
e	从指定地址开始设置各字节单元的值
a	汇编命令
u	反汇编命令
g	运行指定地址开始的代码

R看寄存器，D看内存，E改内存

- **CLC/STC/CMC**:CF置0/置1/取反
- **CLD/STD**:DF置0/置1
- **CLI/STI**:IF置0/置1
- **HLT**:停机
- **NOP**:空转
- **PUSHF/POPF**: 标志出入栈

3.源程序组织

- **JMP NEXT**:跳转执行NEXT语句

3.1.伪指令

- **EQU**: `pi EQU 3.14` 之类的，类似const，定义后值不可变
- 变量定义**DB**系列:
 - `X DB 0`,即char X=0
 - `X DB ?`,即char X, 给X分配一个字节空间，但不赋初值
 - 关于变量类型:

类型	是什么	备注
DB	字节	8bit
DW	字	16bit

还有DD双字，DQ8字节，DT10字节等

注意字是高位存放在高地址，低位存放在低地址

- 重复说明符**DUP**: 类数组
 - `VAR DB 50 DUP (?)`:定义50个字节单元变量（长度为50，无指定内容的char型数组）
 - `STDB DW 100 ("Good!")`:定义100个字单元变量，内容都为"Good!"
 - `VAR DB 10H,77H`，定义一个字节变量，分配两个字节的内存。第一个字节是10H，第二个字节（VAR+1）是77H
 - `VAR DB 5 DUP(4 DUP (0))`:定义一个5*4的二维数组，初始化为0

3.2.标号

- **SEG**:返回段基址，`MOV AX, SEG DATA_S`,这条指令将 DATA_S 段的段地址，以立即数的形式加载到 AX 寄存器中。
- **OFFSET**:。 `MOV AX, OFFSET VAR1` 返回VAR1的偏移量，送入AX

- TYPE:以数字形式返回类型，不是很常用。考试基本不会考FAR

标号及变量类型属性表

	类型属性	类型数字
变量	BYTE	1
	WORD	2
	DWORD	4
标号	NEAR	-1
	FAR	-2



- **LABEL**: 地址别名，在定义阶段，不改变变量实际类型的情况下，用另外一种类型访问
 - 例子：

```
.data
MyBytes LABEL BYTE      ; ① 标签定义
MyData DD 12345678h     ; ② 变量定义（实际数据）
```

这两行是有关联的。LABEL指令会自动catch下一行，MyData是DD类型，对应Mybytes是Byte类型，但两者指向同一块内存。

- **PTR**: 是在使用阶段进行临时的、单次有效的类型强制转换，下一行就不认了，类似 (int)a 这种
 - 指名操作位数，比如 INC BYTE PTR [SI]，表明只针对8位数据+1。如果没有他，这个指令就错误了，因为不确定作用位数
- **LENGTH**: 返回DUP长度，但对嵌套DUP，only最外层
- **SIZE**: 返回总字节数，但有和LENGTH一样的局限

3.3.表达式

- 关系运算符：满足关系返回1，反之返回0

运算符	是什么
EQ	=
NE	≠
LT	<
GT	>
LE	≤
GE	≥

- **HIGH/LOW**: 返回计算结果高8位/低8位
- **地址表达式**:
 - e.g.

```
BVAR DW 1234H,5678H
MOV AX,BVAR+1
```

此时AX寄存器里是7812H，因为低位进低地址，BVAR的内存是|34H|12H|78H|56H|，AX是16位寄存器，对应4位16进制数，因此是7812H

3.3.1.段定义伪指令/代码基本结构

- **SEGMENT/ENDS**:

```
1. 堆栈段
stack0 segment stack
    dw 40h dup(0) ; 分配 40h(64) * 2 = 128 字节堆栈空间，最好用DW(字)类型
stack0 ends
```

2. 数据段

```
data segment ; 定义一个字符串变量, 以 ASCII 0 作为终止符
    Hello db "Hello, World!", 0
data ends
```

3. 代码段

```
code segment ;告诉汇编器段寄存器对应哪个逻辑段
    assume cs:code,ds:data,ss:stack0
main:
    ; 初始化 DS 寄存器, 使其指向 data 段
    mov ax,data ; 将 data 段的地址装入 AX
    mov ds,ax ; 将 AX 的值传给 DS (不能直接 MOV DS, data)
    .....
    mov ah,4ch
    int 21h;这两条类似return 0

    code ends
end main;整个汇编程序结束, 并指定入口为main, 给IP/SP赋值
```

- **格式:** 段名 SEGMENT [对齐类型] [组合类型] ['类别']
 - 只有段名是必须要有的, 其他都是可选项
 - 段名就是stack0/data/code之类的。之所以用stack0是因为stack是关键字, 用ssstack也行
 - 对齐类型: 起始地址必须满足的边界条件,包括BYTE,WORD(被2整除),PARA(被16整除),PAGE(被256整除)
 - 组合类型: 空/PUBLIC/STACK/Common, 貌似用的不多
 - 类别: 'CODE'/'DATA'/'STACK', 用的时候要打单引号, 但也用的不多
- **ASSUME**
assume cs:code,ds:data,ss:stack0 ,建立段名和段寄存器之间的关系
- **END**
 - 标记整个汇编程序的结束
 - 如果后面跟了东西, 比如 END MAIN , 那就是指定程序入口

4.分支与循环程序设计

4.1.无条件转移指令JMP

类似于高级语言 if-else 语句中的 else , 或者 switch 语句中的 default

跳转类型	代码书写形式 (举例)	目的地在哪里?	影响的寄存器	转移范围
段内直接	JMP Label	指令码中给出了 相对位移	仅修改 IP	同一代码段内
段内间接	JMP BX 或 JMP [BX]	目的地在 寄存器或内存单元 中	仅修改 IP	同一代码段内
段间直接	JMP FAR PTR Label	指令码中直接给出了 新的CS和IP	修改 CS 和 IP	跨越不同代码段
段间间接	JMP DWORD PTR [BX]	目的地在 内存双字单元 中 (低位给IP, 高位给CS)	修改 CS 和 IP	跨越不同代码段

因此, 四种方式组合起来就是:

1. **段内直接 (NEAR):** 在同一区域内, 看指令中的指路牌确定偏移。
 2. **段内间接 (NEAR PTR):** 在同一区域内, 根据寄存器或内存中的偏移量确定街道地址。
 3. **段间直接 (FAR):** 跨区域搬家, 指令直接告诉你目标区域和地址的全称。
 4. **段间间接 (FAR PTR):** 跨区域搬家, 指令告诉你去内存中查阅目标区域和地址的完整记录。
- 长短转移:
 - 短转移: ± 127 字节, only in 段内直接
 - 长转移: 大范围

4.2.条件转移指令

条件不一定满足。如果不满足, CPU就执行下一条指令

格式: JX L1 ,JX表示某一跳转指令, L1表示代码段名称

- **JX/JNX:** 这个X代表标志位XF, 比如JZ就是ZF=1跳转, 但JA不是

- JX: 标志位XF=1跳转
- JNX: 标志位XF=0跳转
- 特殊: JE同JZ, JPE是JP,JPO是JNP
- **JCXZ:**
 - CX寄存器值=0跳转
- **JNE:**JUMP IF NOT EQUAL, JE同理
- **无符号数条件转移指令:** (A和B是最近一条CMP/SUB指令编码, **A-B**)
 - JA/JB: JA是A大 (结果>0) 跳转, JB是B大 (结果<0跳转)
 - JAE/JBE: 在JA/JB基础上, 两个相等也跳转
 - JNXX=JX; JNX=JXE, E是Equal, N是Not
- **带符号数条件转移指令:** 【**理解为**补码**】
 - JG/JL: 参考JA/JB
 - N和E同理

4.3.分支程序设计

核心在于跳转逻辑的完整性。汇编不同于高级语言, 他是默认一行一行执行的, 没有高级语言中的大括号作用域.举个例子: 将AL理解为补码, 若AL>0, 则将BL寄存器清零; 反之将其置1.

```
CMP AL,0;将AL和0做比较, 这个比较语句的结果对应的标志位将会被用作下一句的跳转依据
JG L1;补码->有符号数跳转
JMP L2;这句话的作用类似else或者switch语句的default, 兜底作用
```

```
L1: MOV BL,0
    JMP L3;结束这个分支, 作用类似于高级语言的大括号
L2: MOV BL,1
    JMP L3
L3:.....;分支结构出口
```

通过“往回跳转”的思路, 也可以实现循环程序的设计

4.4循环程序设计

- **LOOP系列:** 强制搭配CX寄存器
 - CX--后如果CX≠0就跳转到对应标号
 - LOOPZ/LOOPE还需满足ZF=1
 - LOOPNZ/LOOPNE还需满足ZF=0
 - 诸如:

```
L1:
    .....
    LOOP L1;这样就可以实现循环
```

- 双层循环: 因为LOOP强制绑定CX,而CX寄存器只有一个, 在内层循环过程中当前外层的计数值会被覆盖, 因此需要使用堆栈存储当前外层计数值

5.子程序设计与系统调用

子程序就是函数

- 结构: PROC-ENDP
- 指令:
 - **CALL:** 先将当前的暂停地址 (断点) 压入**堆栈**保存, 然后跳转到子程序的入口地址开始执行。
 - **CALL指令会自动压入两个/四个堆栈单位的内容, 后续子程序内从栈里取参数时要注意**
 - CALL (NEAR PTR)PROC_NAME :段内直接调用,SP-=2存IP
 - CALL BX :段内间接调用,SP-=2存IP
 - CALL (FAR PTR)PROC_NAME :段间直接调用, **SP-=4**以保存IP和CS
 - CALL (DWORD PTR)ADDR :段间间接调用, ADDR是一个双字DD内存单元的逻辑地址, **SP-=4**以保存IP和CS
 - **RET:** return, 回到主程序, 继续断点。
 - RET/RETN :不带参数段内返回, SP+=2,
 - RET N/RETN N : 带参数段内返回, **N为偶数**, SP+=(2+N),不怎么考, 看一看就行了

- 另外两个段间同理，就是把CALL反向执行。写作RET/RETF
- 示例：注：子程序设计必须使用堆栈段，因此必须定义STACK SEGMENT

```
子程序名  PROC  [NEAR / FAR]    ; 段间/间接必须指定段内(NEAR)或段间(FAR)属性, NEAR可以省略
...      ; 子程序主体代码
RET      ; 返回指令
子程序名  ENDP                  ; 结束定义
```

5.2.系统调用

- INT/IRET：中断/中断恢复。
 - INT中SP-=6，入栈存储FR,CS,IP，清零IF/TF
 - IRET将INT操作完全镜像
- INT的使用：

```
MOV AH,XXH ;往AH寄存器中放一个功能号
INT 21H    ;调用 DOS 系统服务：按XXH号功能执行
```

- 功能号：
 - **01H**：单个字符输入，ASCII保存进AL
 - **02H**：单个字符输出，来自DL
 - **0AH**：字符串输入，在MOV和INT中间要插一行 MOV DX, 缓冲区起始偏移量 或者 LEA DX, 缓冲区变量，输入直到读取了回车，对输入缓冲区格式有特殊要求：[^0fa57b](#)
 - **09H**：字符串输出，在MOV和INT中间要插一行 MOV DX, 缓冲区起始偏移量 或者 LEA DX, 缓冲区变量，输出直到\$（这个符号本身不输出，但所有走09H输出的字符串都必须以\$结尾）
 - 此时系统逻辑：查看 AH：哦，你要打印字符串。查看 DX：哦，字符串在内存的这个位置。开始干活：从 DS:DX 指向的内存单元开始，一个字节一个字节地把字符送到屏幕上。
 - **4CH**：结束当前程序并返回 DOS。

6.数值运算程序设计

- BCD码
 - 非组合BCD：一个字节存1个十进制数，低4位为BCD码，高4位置0或写为0011B表示ASCII
 - 组合BCD：一个字节存2个十进制数
- BCD加减：计算机运算用二进制，表示用16进制，而我们把结果看成BCD码的十进制，因此部分情况需要加6/减6。下面是加法，减法同理：
 - 低4位结果为0-9，且AF=0：无需有新动作
 - 低4位结果为0-9，且AF=1：计算结果+6才是正确结果，
 - 比如1001(9)+1000(8)=1 0001=11H->11(看作十进制)，但正确结果是17，因此要加6
 - 低4位大于9（A-F）：计算结果+6才是正确结果
 - 比如1001(9)+0001(1)=1010=0AH->0AH+6=10H->10
 - 加法结果可能比加数高一位，定义结果变量时需注意
 - 修正回十进制的指令：
 - **AAA/DAA**，非组合加法/组合加法
 - **AAS/DAS**，非组合减法/组合减法
 - **AAM**：非组合乘法
- BCD乘除：
 - 乘法：MUL(无符号数)/IMUL(带符号数)，后跟SRC， $AX(AL)* = SRC$ ，用**AAM**指令修正回十进制
 - 除法：DIV(无符号数)/IDIV(带符号数)，后跟SRC
 - 字节除法：商放AL，余数放AH
 - 字除法：商放AX，余数放DX
- 有符号数扩展：正负性不变
 - CBW：字节->字
 - CWD：字->双字

7.串操作

源串存储于DS:[SI]，目标串存储于ES:[DI]

- **REP**: Repeat, 无条件重复直到CF=0
 - REPE/REPZ: 相等时重复, 即CF=0&&ZF=1时重复
 - REPNE/REPNZ: 不相等时重复
- **MOVS**: 数据从源串复制到目标串, DS:[SI]-> ES:[DI], 每一次只能拷贝一个地址下的单个Byte, 如果要拷贝整个串需要 REP MOVS
- **STOS**: 串存储, AL/AX->ES:[DI]
- **LODS**: 串读取, DS:[SI]->AL/AX
- **CMPS**: 串比较, 即 MOV AL/AX,DS:[SI] & CMP AL/AX,ES:[DI]
- **SCAS**: 在目标串中查找累加器AL/AX的值, 即 CMP AL/AX,ES:[DI]

X.补充

- \$ 标号表示当前地址, LEN EQU \$-STR 就能求得STR字符串的长度, 遍历时需要
- ORG 指令: 强行将地址计数器DX设置为后面表达式计算出来的结果
- 如果写程序过程中要用数组: 用SI代表i, 先 LEA SI,数组 , 然后指定循环次数进CX, 把[SI]即数组第一个数送入变量 (可以是AL), i++就是 INC SI。为什么要用AL这种变量, 是因为MOV不能有两个内存操作数, MOV VAR,[SI] 是不行的。
- **END系列**
 - END 是整个汇编程序的结尾, 比如 END MAIN
 - ENDS 是段结束, 用于段定义, 比如 CODE ENDS
 - ENDP 是子程序结束, 和 PROC 搭配, 比如 FUNC ENDP
- OF:
 - 位移次数为1的移位指令中, 如果最高位变化, 那OF=1
 - 循环移动中, OF=新CF异或新MSB
 - 加减法中, 出现不符合常理的情况就是1, 比如正数+正数=负数
- AAA指令:
 - 如果AL低4位>9,或者AF=1: +6且AF=CF=1
 - 反之, AF=CF=0

X.1.代码改错题常考类型

1. 双操作数指令最多有一个内存操作数, 两个操作数不能同时为存储器寻址, 比如 ADD [BX], 100H[SI] , MOV VAR,[SI]
2. 单操作数指令不能有立即数, 比如 PUSH 65FFH
3. 位数不匹配, 比如 AND AL,BX , 前者8位, 后者16位; 或者 ADR1 DB VAR1,DB是8位, 但汇编地址都是16位
4. 位数不确定, 比如 SUB [BX], 64H , 必须明确指出大小, 如 SUB BYTE PTR [BX], 64H 或 SUB WORD PTR [BX], 64H
5. 不能把立即数直接给段寄存器, 也不能段寄存器间/立即数间传送, 比如 MOV DS,11H 或者 MOV DS,CS 或者 MOV VA1,VA2
6. 条件转移指令只能跟标号, 不能跟寄存器, 比如 JC BX
7. 两个变量名不能直接相加, 比如 ADR1 DW VAR1+VAR2 ,因为变量名背后是地址偏移量, 相加没有数学意义
8. 指令助记符 (关键字) 冲突, 比如 ADR1 EQU SUB , SUB就不能用作自定义变量

X.2.汇编代码撰写示例

A.子程序

- 题目: 设计一个汇编子程序统计指定的8位补码数组中负数的个数。入口参数包括指定数组偏移量, 长度, 使用堆栈传递。出口参数为负数个数, 使用AL传递。
- 思路:
 - 1.统计8位补码数组中负数个数->统计有多少个数符号位为1-> TEST NUM,10000000B 或者 CMP NUM,10000000B

```
COUNT_NUM PROC
PUSH BP ;为什么要这个看下一行
MOV BP,SP ;SP默认指向栈顶,但不能[SP]访问堆栈,因此要用BP访问堆栈。自然需要先把BP原内容存档(入栈)

MOV SI,[BP+4] ;传递参数: 偏移量
MOV CX,[BP+6] ;传递参数: 数组长度
;为什么是+4/+6?因为在子程序CALL之前,栈内会塞入上面两个参数,然后CALL过程中会压入IP,最后在子程序内再压入BP存档,因此需要+4/+6访问参数

XOR AL,AL ;首先要把计数器AL清零
LOOP1:
```

```

MOV DL,[SI] ;SI本身是内存地址，而非地址里的值，因此要先从内存里把值取出来放进CPU的数据寄存器DX中
TEST DL,80H ;80H=10000000B,Check首位是否为1，如果为1，ZF=0，反之亦然
JZ L1
INC AL ;计数器AL++
L1:
INC SI ;数组指针++，访问数组中下一个元素
LOOP LOOP1 ;循环回去，这里会自动DEC CX，不需要再写
POP BP
RET 4 ;此时堆栈里还有偏移量和数组长度两个参数，需要把他们用RET清理掉以保证；当然在主函数中再清理也OK，因此这里写RET也对

```

B.INT 21H DOS功能

^0fa57b

0AH：字符串输入，需要DX寄存器提供缓冲区偏移量地址，而缓冲区结构是有特定要求的，在数据段定义的时候就要注意：

+0 最大可输入字符数，下面代码里的MAX_LEN

+1 实际输入字符数（输入时的计数器，初始化为0），代码里的ACT_LEN

+2 输入的字符内容开始，代码里的IN_STR

代码示例：

```

DATA SEGMENT
    MAX_LEN DB 50 ;最多输入50个字符
    ACT_LEN DB 0 ;当前输入0个
    IN_STR DB 51 DUP(0) ;最后还有一个回车代表输入结束
DATA ENDS

CODE SEGMENT
.....
    MOV AH,0AH ;输入中断向量号
    LEA DX MAX_LEN ;把缓冲区偏移地址赋给DX寄存器,或者写成MOV DX,OFFSET MAX_LEN也行
    INT 21H ;启动DOS服务

```

X.3. ASCII

- 数字字符：30H~39H
- 大写字母：41H开始
- 小写字母：61H开始
- 回车字符：0DH
- 换行字符：0AH

X.4.标志位汇总

- INC/DEC不影响CF
- AND/OR/XOR系列位指令不影响AF，强制将CF/OF清零
- AAA指令如果+6就把AF和CF置1，反之置0
- 不影响任何标志位：
 - 移动类指令，比如MOV,POP,PUSH,XCHG,LEA系列
 - NOT
 - 循环控制LOOP系列
 - 跳转控制JX系列，包括CALL和RET
 - WAIT和NOP（空操作）
- 会产生不确定标志位：(不改变标志位的也是不确定，因为你不知道这个标志位之前到底是0还是1)
 - 乘除法指令，乘法只有CF和OF确定，除法所有都不确定
 - 移位指令，比如SHL,SHR,SAL,SAR，在移位次数大于1时OF不确定
 - INC/DEC：CF不变，不由指令具体运算确定，故称不确定
 - 串操作系：同INC/DEC

X.5.寄存器隐含使用汇总

指令	隐含寄存器	作用
MUL / DIV	AX (或 DX:AX)	存放被乘数、被除数、结果
IN / OUT	AX (或 AL)	数据中转站
XLAT	BX, AL	表头地址，索引/结果
LOOP / REP	CX	计数器
移位 (CL)	CL	移位次数
IN / OUT (DX)	DX	端口地址 (>255)
MOVS 等	SI, DI, ES	源/目指针，目的段
PUSH / POP	SP, SS	栈顶指针，栈段